Trabalho de Conclusão de Curso de Engenharia da Computação

**Protótipo para detecção, localização e seleção automática de ação por sensoriamento**

# Diagrama de Implantação

Gabriel Vasconcellos de Morais – Matheus Emanoel Gomes Souza

Orientador: Anker Douglas Loss

Curso: Engenharia da Computação — Faculdade do Centro Leste (UCL)

## 1 Apresentação

Este documento especifica o diagrama de implantação do sistema **RadarTorres**, descrevendo a distribuição física dos componentes de software sobre os nós de hardware envolvidos: o computador do usuário, onde roda a aplicação desktop, e o Arduino, responsável pela leitura dos sensores e pelo acionamento das torres demonstrativas. Complementa o *Diagrama de Classes* (estrutura estática) e o *Modelo de Banco de Dados* (persistência), mostrando onde cada parte do sistema efetivamente executa e como os nós se comunicam.

O sistema representa apenas dois nós físicos reais — computador do usuário e Arduino —, sem nenhum componente de servidor, web ou nuvem: trata-se de uma aplicação desktop única e autocontida, comunicando-se com um único microcontrolador externo via porta serial.

## 2 Nós e Elementos

Tabela 1: Nós e elementos do diagrama de implantação

| Nó | Elemento | Papel |
|---|---|---|
| Computador do Usuário (Windows 10/11, 64-bit) | `RadarTorres.App` (.exe) | Único processo, self-contained (.NET 9), instalado por `installer/RadarTorres.iss`. |
| ↪ dentro do processo | Apresentação (`Views` + `ViewModels`) | Captura interação do operador, *binding* declarativo (`ViewModelBase`, `RelayCommand`). |
| ↪ dentro do processo | Serviços (regra de negócio) | `ISerialCommunicationService`, `ITargetTrackingService`, `ITowerSelectionService`, `IFireControlService`, `IZonaMortaService`, `IAuthService`, `IPermissionService`. |
| ↪ dentro do processo | Persistência local | Leitura/escrita do banco SQLite via `Microsoft.Data.Sqlite` (`SqliteConnectionFactory` / `Sqlite*Repository`). |
| ↪ dentro do processo | .NET 9 Desktop Runtime | Embutido pelo instalador — sem dependência externa a instalar. |

Tabela 1 (continuação)

| Nó | Elemento | Papel |
|---|---|---|
| Arduino (microcontrolador) | Firmware | Lê sensores, envia leituras de alvo, recebe comandos e aciona as torres demonstrativas. |
| Armazenamento local (disco do usuário) | `%AppData%\RadarTorres\Data\radartorres.db` (SQLite) | Usuários, auditoria, objetos detectados, preferências e demais dados cadastrais; CSV permanece apenas como exportação manual da tela de Objetos Detectados. |
| Sensores de detecção | — | Ângulo + distância, alimentam o Arduino. |
| Torres demonstrativas / indicador | — | Laser de baixa potência ou LED, nunca armamento real. |

# 3 Diagrama de Implantação

O diagrama representa apenas dois nós físicos reais: o computador do usuário e o Arduino, conectados exclusivamente por USB/Serial. Fonte: `Implantacao_RadarTorres.puml` (PlantUML), nesta mesma pasta.

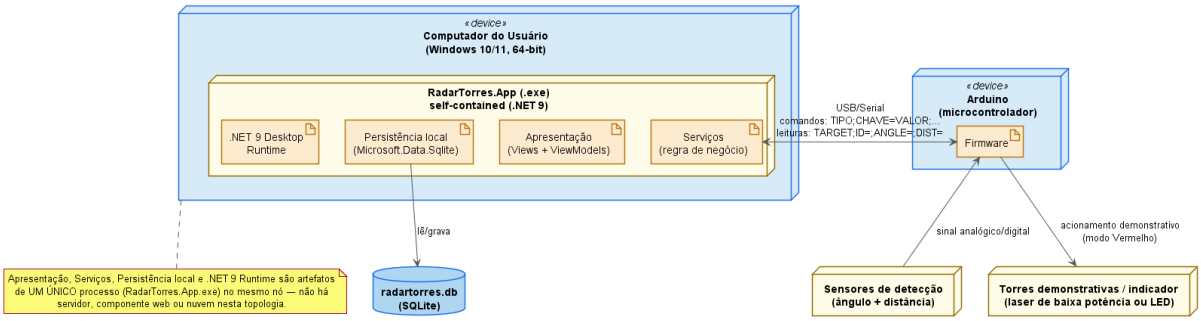



Figure 1: Diagrama de implantação do RadarTorres.

# 4 Protocolo de Comunicação (USB/Serial)

O único canal entre os dois nós físicos é a porta serial sobre USB, com mensagens em texto simples (ver `Docs/Tecnica/COMUNICACAO_ARDUINO.md` para a especificação completa) — Tabela 2.

Tabela 2: Mensagens trocadas no link serial

| Direção | Formato | Finalidade |
|---|---|---|
| Computador → Arduino | `TIPO;CHAVE=VALOR;...` | Comandos (troca de modo, acionamento, configuração) |
| Arduino → Computador | `TARGET;ID=;ANGLE=;DIST=` | Leituras de alvo (ângulo + distância detectados) |

Não há reconhecimento criptográfico nem autenticação no link serial — a segurança do sistema depende do controle de acesso físico ao cabo USB e do fato de o Arduino ser um periférico dedicado, sem acesso a rede.

# 5 Explicação

O diagrama usa a notação UML padrão de implantação: `Computador do Usuário` e `Arduino` são nós físicos marcados com o estereótipo **«device»**; dentro do nó do computador, o processo `RadarTorres.App.exe` é um nó de ambiente de execução que contém quatro **«artifact»** (Apresentação, Serviços, Persistência local, .NET 9 Runtime) — artefatos de software do mesmo processo, não componentes distribuídos em máquinas ou processos separados. O banco de dados (`radartorres.db`) é representado com o ícone de cilindro (**database**), reservado para dados persistidos pelo sistema. Os dois acoplamentos mais relevantes do processo saem da caixa para fora: Serviços comunica-se com o Arduino pela porta serial, e a camada de Persistência lê/grava o banco SQLite local em disco.

Do lado do Arduino, o Firmware concentra as quatro responsabilidades físicas do dispositivo: ler os sensores de ângulo/distância, enviar leituras de alvo ao computador, receber comandos e acionar as torres demonstrativas.

A escolha por um único processo desktop self-contained, sem servidor, é coerente com o argumento de segurança/leveza já discutido para o SGBD (ver documento *Modelo de Banco de Dados*): menor superfície de ataque (nenhuma porta de rede exposta), zero administração de infraestrutura e instalação totalmente local, adequadas ao perfil de nó único do protótipo.

# 6 Conclusão

O diagrama de implantação evidencia a natureza autocontida do RadarTorres: um único nó de software (a aplicação desktop, self-contained sobre o .NET 9) comunicando-se com um único microcontrolador externo via porta serial, sem qualquer dependência de infraestrutura de servidor, web ou nuvem. Toda a persistência ocorre localmente, no próprio computador do usuário, via SQLite, e a segregação em blocos internos (Apresentação/Serviços/Persistência/Runtime) é apenas didática — não representa componentes fisicamente distribuídos.

# 7 Validação

Nós, artefatos e protocolo confirmados em `installer/RadarTorres.iss`, `App.xaml.cs` e `Docs/Tecnica/COMUNICACAO_ARDUINO.md`. Driver de persistência (`Microsoft.Data.Sqlite` 9.0.0) confirmado em `src/RadarTorres.App/RadarTorres.App.csproj:50`. A segregação em quatro blocos internos (Apresentação/Serviços/Persistência/Runtime) é didática, não uma leitura de componentes fisicamente distribuídos — ver Seção 6 de `UML_RadarTorres.md` (pasta `UML/`).